

KeynetikPOS

Engineering & Technical Documentation

Architecture • Data Model • Modules • Build • Security

Version 2.0.0

Qt 6 / C++17 • SQLite • CMake • Windows (MinGW / MSVC)

About this document

This is the technical reference for KeynetikPOS, intended for engineers building, extending or maintaining the system. It describes the architecture, the persistence layer and schema, module responsibilities and collaborations, the critical transaction flows, the concurrency/threading model, the security implementation, and the build & deployment pipeline. It closes with known engineering trade-offs and technical debt.

Property	Value
Language / standard	C++17
UI framework	Qt 6 (Widgets, Sql, PrintSupport, Network, SerialPort)
Persistence	SQLite (SQLITE driver), single embedded file
Build system	CMake $\geq$ 3.16 (AUTOMOC / AUTOUIIC / AUTORCC)
Toolchains	MinGW (Qt 6.11) and MSVC 2022
Packaging	windeployqt + CPack / NSIS installer
Source size	~21,000 LOC across ~70 translation units / headers
Target OS	Windows (Win32 GUI subsystem)

Contents

- 1 System Architecture
- 2 Source Layout & Module Map
- 3 Persistence Layer & Data Model
- 4 Core Domain Types
- 5 Checkout Transaction Flow
- 6 Subsystem Design Notes
- 7 Concurrency & Threading Model
- 8 Security Implementation
- 9 Build, Packaging & Deployment
- 10 Configuration & Runtime Environment
- 11 Engineering Notes & Technical Debt
- 12 Extension Guide

1. System Architecture

KeynetikPOS is a single-process, single-binary desktop application. It follows a loosely layered design: a Qt Widgets presentation layer, a set of manager/service objects holding business logic, and a thin data-access layer over SQLite. Several cross-cutting services (database, authentication, licensing) are exposed as process-wide singletons.

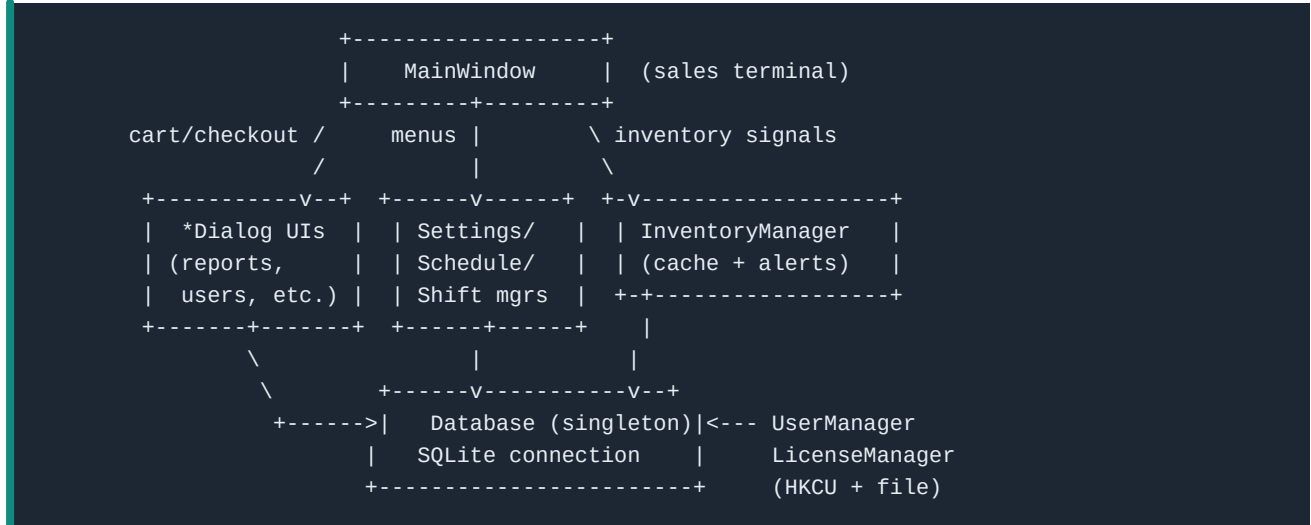
Layered view

Layer	Responsibility	Representative types
Presentation	Windows, dialogs, widgets, theming, input	MainWindow, *Dialog, enhancedanalytics widgets, ColorScheme
Application / logic	Business rules, orchestration, validation	UserManager, InventoryManager, ShiftManager, ScheduleManager, SettingsManager, ReceiptPrinter, LicenseManager
Data access	SQL, schema, migrations, transactions	Database (singleton), per-manager table owners
Infrastructure	OS / hardware / network I/O	BarcodeReader (serial/HID), Qt Network (messaging, licensing), QPrinter

Design patterns in use

Pattern	Where	Why
Singleton (Meyers)	Database, UserManager, LicenseManager, ThemeManager	One DB connection / one auth context / global theme
Strategy	BarcodeReader (HID vs serial); ReceiptPrinter (thermal/standard/PDF)	Swap behaviour behind one interface
Observer (signals/slots)	InventoryManager, ScheduleManager, managers→UI	Decouple producers of events from the UI
Abstract factory / registry	MessageProvider + ScheduleManager registry	Add SMS/WhatsApp providers without touching the engine
Value object / DTO	Product, Sale, Receipt, BusinessSettings, ShiftRecord	Plain structs move data across layers
Active record-ish DAO	Database + per-manager table owners	Typed query methods return domain structs

Component dependency sketch



Startup sequence (main.cpp)

```
QApplication a(argc, argv);
AntiDebug::isThreatDetected()      -> abort if debugger (release only)
LicenseManager::instance().initialize() // offline, non-blocking
    - tamper check -> trial/limited/full state
Database::instance().initialize() // open, PRAGMA, create+migrate, seed
LoginDialog (<=3 attempts) -> UserManager::login()
    - if must_change_password -> forcePasswordChange()
MainWindow w; w.show();
QTimer::singleShot(2000, startOnlineHeartbeat) // async revalidation
return a.exec();
```

Singletons

Database, UserManager and LicenseManager use the Meyers singleton (function-local static). This gives global access and a single DB connection, but couples callers to the instances and is the main obstacle to unit testing (see §11).

2. Source Layout & Module Map

All translation units live at the project root and are listed explicitly in CMakeLists.txt. Modules pair a .cpp/.h by feature.

Module	Files	Role
Entry point	main.cpp	Bootstrap, license/login gating, password enforcement
Main window	mainwindow.*	Sales terminal, cart UI, menu wiring, checkout
Database	database.*	Schema, migrations, products/sales/refunds/stock DAO
Auth / RBAC	usermanager.*, user.*, logindialog.*, usermanagementdialog.*	Login, hashing, permissions, user CRUD UI
Inventory	inventorymanager.*, inventorydialog.*, lowstockdialog.*	Stock cache, alerts, catalogue UI, CSV import/export
Stock control	stockmanager.*	Restock/adjust UI with audit
Discounts	discount.*, discountdialog.*	Discount engine, presets, PIN, log
Payments	paymentmanager.*, paymentdialog.*	Tender capture, change, records
Receipts	receiptprinter.*	Thermal / standard / PDF rendering
Shifts	shiftmanager.*, shift_dialogs.*, shift_type.h	Shift lifecycle, cash
Reports	reportsdialog.*, zreportdialog.*, saleshistorydialog.*	Sales reports, Z-report, history + refunds
Analytics	analyticsmanager.*, analyticsdashboard.*, analyticsdialog.*, enhancedanalytics.*	Aggregation + chart widgets
Messaging	schedulemanager.*, messageprovider.*, whatsappmanager.*, Africataalkingprovider.h, scheduledialog.*, scheduleeditordialog.*	Provider registry + scheduling engine
Barcode	barcodereader.*, barcodescannermanager.*	HID/serial input
Settings	settingsmanager.*, settingsdialog.*	Config persistence + UI
Licensing	licensemanager.*, antidebug.h	Trial, activation, tamper, anti-debug
Shared	passwordhasher.h, colorscheme.*, CartItem.h	Crypto helper, theme, cart item

3. Persistence Layer & Data Model

A single SQLite file holds all state, created at `QStandardPaths::AppDataLocation/pos_database.db`. The Database singleton owns the default connection; `PRAGMA foreign_keys = ON` is set per-connection at open. Schema is created idempotently with `CREATE TABLE IF NOT EXISTS`, and additive column changes are applied by an `ensureColumn()` migration helper (since `IF NOT EXISTS` never alters existing tables).

Core schema (database.cpp)

Table	Key columns	Notes
categories	id PK, name UNIQUE, description	Lookup for product grouping
products	id PK, name, category FK→categories(name), price, cost_price, profit_margin, stock_quantity, barcode UNIQUE, is_active	Soft-deleted via is_active
sales	id PK, sale_date, subtotal, tax, discount, total, payment_method, amount_paid, change_due	amount_paid/change_due added by migration
sale_items	id PK, sale_id FK, product_id FK, product_name, quantity, price, cost_price, subtotal	Denormalised name + cost for historical accuracy
users	id PK, username UNIQUE, password_hash, full_name, email UNIQUE, role, is_active, last_login, created_by, must_change_password	role CHECK constraint
user_activity_log	id PK, user_id FK, username, action, details, timestamp	Append-only audit trail
stock_adjustments	id PK, product_id FK, product_name, change_qty, old_qty, new_qty, reason, adjusted_by, adjusted_at	Stock movement audit
refunds	id PK, sale_id FK, refund_date, total_refunded, reason, processed_by	One refund per sale enforced in code

Module-owned tables

Table	Owner	Purpose
AppSettings	SettingsManager	Key/Value config store
Shifts	ShiftManager	Shift lifecycle, floats, totals
DiscountLog	DiscountManager	Applied-discount audit
PaymentRecords	PaymentManager	Tender records
message_schedules	ScheduleManager	Automated report schedules
ProductBarcodes / ScanHistory	BarcodeScannerManager	Barcode map & scan log

Indexing

Indexes back the hot audit/login paths: `idx_users_username`, `idx_users_email`, `idx_activity_log_user / _timestamp`, `idx_adj_product / _time`.

Representative DDL

The two central sales tables, verbatim from database.cpp:

```
CREATE TABLE IF NOT EXISTS sales (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  sale_date     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  subtotal      REAL NOT NULL,
  tax           REAL DEFAULT 0,
  discount      REAL DEFAULT 0,
  total         REAL NOT NULL,
  payment_method TEXT DEFAULT 'Cash',
  amount_paid   REAL DEFAULT 0,  -- added via ensureColumn()
  change_due    REAL DEFAULT 0); -- added via ensureColumn()

CREATE TABLE IF NOT EXISTS sale_items (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  sale_id       INTEGER NOT NULL,
  product_id    INTEGER NOT NULL,
  product_name  TEXT NOT NULL,  -- denormalised snapshot
  quantity      INTEGER NOT NULL,
  price         REAL NOT NULL,
  cost_price    REAL NOT NULL DEFAULT 0,  -- for profit calc
  subtotal      REAL NOT NULL,
  FOREIGN KEY (sale_id) REFERENCES sales(id),
  FOREIGN KEY (product_id) REFERENCES products(id));
```

Relationships & integrity

- sales 1–N sale_items; products 1–N sale_items; sales 1–1 refunds (one refund per sale, enforced in code via isRefunded()).
- users 1–N user_activity_log; products 1–N stock_adjustments.
- products.category is a **text** FK to categories(name) (a non-PK column) — renaming a category can orphan products; flagged in §11.
- Denormalisation is intentional: sale_items snapshots product_name and cost_price so historical sales and profit survive later product edits or deletions.

Database API surface (database.h)

Group	Methods
Products	getAllProducts, getProductsByCategory, getProductById, getProductByBarcode, addProduct, updateProduct, deleteProduct (soft), getAllCategories
Stock	updateStock, decreaseStock, increaseStock, getStock, adjustStockWithLog, logStockAdjustment, getStockHistory
Sales	recordSale (atomic), getAllSales, getSalesByDateRange, getSaleItems, getSaleById
Analytics	getTotalSalesToday / ThisMonth, getTotalTransactionsToday, getTopSellingProducts, getActualGrossProfit[Today/ThisMonth]
Refunds	processRefund (atomic, restocks + logs), isRefunded
Infra	initialize, isOpen, ensureColumn, executeQuery, getLastError

4. Core Domain Types

Plain structs carry data between layers; richer behaviour lives on managers.

Type	Defined in	Purpose
Product	database.h	Catalogue row; static calculateSellingPrice(cost, margin)
Sale / SaleItem	database.h	Sale header and line items
CartItem	CartItem.h	In-memory cart line with getSubtotal()
Cart	mainwindow.h	One open transaction: items, discount, timestamps
Receipt	receiptprinter.h	Render model for a printed/exported receipt
User / UserRole / Permission	user.h	Identity + RBAC enums; RoleManager maps role→perms
BusinessSettings	settingsmanager.h	All configurable settings as a value object
ShiftRecord	shift_type.h	Shift state and running totals
MessageSchedule	MessageSchedule.h	Schedule definition + type/report enums
ZReportData	zreportdialog.h	Computed Z-report model
DiscountResult / DiscountPreset	discount.h	Discount computation outputs

RBAC model

RoleManager::hasPermission(role, perm) is a pure function: Admin is allowed everything; Manager, Cashier and Viewer are explicit allow-lists over the Permission enum (18 permissions spanning sales, inventory, reports, users, system). UI affordances are enabled/disabled from these checks, and every privileged manager method re-checks server-side.

5. Checkout Transaction Flow

Checkout is the most critical path. It is fully atomic: the sale header, all line items and stock decrements commit together or not at all.

Sequence

```
MainWindow::onCheckout()
1. computeCartTotals(subtotal, discount, settings)
   tax on (subtotal - discount); inclusive or exclusive
2. PaymentDialog -> method, amountPaid, change
3. build QVector<SaleItem> from cart
4. Database::recordSale(items, totals, method, paid, change):
   db.transaction()
   for each item: SELECT stock FOR validation (oversell guard)
   INSERT INTO sales ... -> saleId
   for each item:
     INSERT INTO sale_items ...
     UPDATE products SET stock_quantity = stock_quantity - qty
   db.commit() (rollback on ANY failure, returns -1)
5. on success: refresh inventory cache, print receipt,
   log action, record to shift, reset cart
6. on failure: keep cart, show error, print nothing
```

Invariants enforced

(a) stock never decremented more than once per unit sold; (b) a sale row never exists without its line items; (c) stock cannot go negative under concurrent sales because validation and decrement share one transaction.

Money handling

Amounts are double / SQLite REAL. A `roundCents()` helper (round to 2 dp) is applied at totals boundaries to contain floating-point drift. Integer-cents storage is a recommended future change (see §11).

Totals algorithm (`computeCartTotals`)

```
base = subtotal - clamp(discount, 0, subtotal)
if !taxEnabled || taxRate <= 0:
  tax = 0; total = base
elif taxInclusive: # price already contains tax
  tax = base - base / (1 + taxRate)
  total = base
else: # tax added on top
  tax = base * taxRate
  total = base + tax
# all three lines rounded to cents
```

`recordSale` return semantics

Result	Meaning	Side effects
≥ 1	Success; value is the new sale id	sales + sale_items inserted, stock decremented, committed
-1	Failure (empty cart, oversell, insert error, commit error)	Full rollback; <code>getLastError()</code> set; cart preserved; no receipt

Refund flow (`Database::processRefund`) — also atomic

```
db.transaction()  
  sale = getSaleById(id);  abort if not found  
  INSERT INTO refunds (sale_id, total_refunded, reason, processed_by)  
  for each sale_item:  
    increaseStock(product_id, qty)          // return to stock  
    logStockAdjustment(... 'Refund for Sale #id' ...)  
db.commit()  // rollback on any failure
```

Guard

The Sales History UI calls `isRefunded()` before allowing a refund and disables the button afterwards, so a sale cannot be refunded twice.

6. Subsystem Design Notes

InventoryManager

- Holds an in-memory `QMap<int, InventoryInfo>` cache, refreshed on a `QTimer` and after each sale.
- Status thresholds: 0 = out, 1–4 critical, 5–20 low, >20 healthy.
- Emits `inventoryLow` / `Critical` / `OutOfStock` / `Updated` / `Restocked` signals consumed by the main window and dialogs.
- Post-fix, it no longer issues its own sale decrement — `recordSale` owns stock mutation; the manager only syncs cache and emits warnings.

ScheduleManager & providers

- Provider registry over a `MessageProvider` interface; concrete providers: `AfricasTalkingProvider` (SMS) and `WhatsAppManager` (Twilio).
- `QTimer` tick evaluates each schedule's `shouldFire()` against `type/time/day/threshold`.
- Report bodies are built from the shared Z-report text generator; sends are async via Qt Network with `delivered/error` signals; schedules persist in `message_schedules`.

BarcodeReader

- Strategy by mode: `keyboard-wedge` installs an application event filter and buffers keystrokes with an inter-character timeout; `serial` mode reads a `QSerialPort`.
- Configurable port, baud, prefix/suffix stripping; emits `barcodeScanned(QString)`.

ReceiptPrinter

- Three strategies behind one API: `thermal` (plain text), `standard` (HTML via `QTextDocument/QPrinter`), and `PDF` export.
- Caches the last receipt for reprint; company header and footer injected from settings.
- Thermal layout is a fixed 42-column monospace render (centered header, padded item/qty/total columns, '=' / '-' rules) suited to roll printers.

Discounts & payments

- `DiscountManager` exposes `applyPercentage` / `applyFixedAmount` returning a `DiscountResult` (type, value, computed amount, label, appliedBy) and logs to `DiscountLog` with the shift id and cart id.
- Built-in presets: 5/10/15/20% off, Staff 25%, and \$2/\$5/\$10 fixed-amount off.
- `PaymentManager` models 8 tender types (`Cash`, `Card`, `MobileMoney`, `BankTransfer`, `Check`, `GiftCard`, `StoreCredit`, `Other`) with per-method name/icon/colour and daily/shift summaries; the live checkout stores `method + amount_paid + change_due` on the sale.

Theme system (ColorScheme / ThemeManager)

- `ThemeManager` is a singleton holding the active `ColorScheme` (a flat bag of `background/text/border/semantic/button-state` colours).
- Dialogs pull colours from the scheme so light/dark mode is consistent; the choice is persisted in `QSettings` and toggled from the Settings menu.

Analytics query catalogue

AnalyticsManager (and Database) compute metrics directly in SQL over sales / sale_items. Key building blocks:

Metric	Approach (SQL essence)
Today / month revenue	SUM(total) filtered by DATE(sale_date)=DATE('now') or strftime('%Y-%m', ...)
Transactions today	COUNT(*) over sales for the current date
Top sellers	SUM(quantity) grouped by product_name, ORDER BY DESC LIMIT n
Actual gross profit	SUM((price - cost_price) * quantity) joining sale_items→sales over a date range
Category breakdown	Revenue grouped by product/category with % of total
Hourly heatmap	Revenue / count bucketed by hour-of-day (strftime '%H')
Peak hours	Ranked hour buckets with a recommended trading window
Dashboard summary	today/yesterday/week/month revenue, avg ticket, top product/category, growth rate

Richer typed structs (DailySalesStats, HourlySalesStats, ProductSalesStats, CategoryStats, PeakHourInfo, DashboardSummary) carry results to the chart widgets in enhancedanalytics, which render without QtCharts (custom-drawn widgets).

7. Concurrency & Threading Model

- The application is effectively single-threaded: all UI and business logic run on the Qt GUI (main) thread driven by the event loop.
- There are no worker threads, mutexes or shared-state locks; the single SQLite connection is only touched from the GUI thread, which is the supported usage.
- Network I/O (licensing heartbeat, SMS/WhatsApp sends) is asynchronous and non-blocking via QNetworkAccessManager signals — the UI never stalls on a socket.
- Time-based work (inventory refresh, schedule ticks, datetime clock, the post-startup license heartbeat) is driven by QTimer callbacks on the same event loop.
- Hardware input (serial barcode) arrives via QSerialPort readyRead signals; HID input via the event filter — both on the GUI thread.

Implication

Any future long-running operation (large CSV import, bulk report generation, blocking DB work) should be moved off the GUI thread (QtConcurrent / QThread + a dedicated DB connection) to keep the terminal responsive.

Key signal / slot wiring

Emitter — signal	Consumer — slot / effect
InventoryManager::inventoryLow/Critical/OutOfStock	MainWindow alert handlers + status toasts
InventoryManager::inventoryUpdated	Product grid / totals refresh
SettingsManager::settingsChanged	MainWindow refreshTaxTitle + updateTotals; receipt header reload
ScheduleManager::messageDelivered / messageError	Surface send success/failure
BarcodeReader::barcodeScanned	Resolve product → addToCart
ShiftManager::shiftOpened / shiftClosed	Update shift UI and Z-report inputs
QTimer (inventory / schedule / clock / heartbeat)	Periodic refresh, dispatch, revalidation

Error-handling conventions

- DAO methods return bool / -1 sentinels and stash a message in Database::lastError() (getLastError()); the UI surfaces it via QMessageBox.
- Multi-statement mutations (recordSale, processRefund, deleteUser) wrap work in db.transaction() / commit() with rollback on any failure.
- Network calls degrade gracefully: timeouts/aborts set serverReachable=false rather than blocking or crashing.
- Diagnostic logging uses qDebug/qWarning/qCritical; user actions go to the user_activity_log audit table.

8. Security Implementation

Credential hashing (passwordhasher.h)

- Salted **PBKDF2-HMAC-SHA256**, 100,000 iterations, 16-byte random salt (from QRandomGenerator::system()), 32-byte derived key, via QPasswordDigestor::deriveKeyPbkdf2.
- Constant-time comparison (XOR-accumulate) to resist timing attacks.
- Legacy unsalted SHA-256 hashes still verify and are transparently re-hashed to PBKDF2 on next successful login (needsRehash()).
- Used for both user passwords and the discount PIN.

Stored hash format

```
pbkdf2-sha256 $ <iterations> $ <salt-hex> $ <derived-key-hex>
example:
pbkdf2-sha256$100000$9f2c...b1$4d6a...e0

verify(secret, stored):
if not startswith 'pbkdf2-sha256$': legacy SHA-256 path
split into 4 parts -> iters, salt, expected
key = PBKDF2(SHA256, secret, salt, iters, len(expected))
return constantTimeEquals(key, expected)
```

Account policy

- Default seed admin/admin123 carries `must_change_password = 1`; admin-issued resets set it too, forcing a new password before entry.
- Login is capped at 3 attempts; all auth events are audited.

Licensing & tamper resistance (licensemanager.cpp)

- State in HKCU; integrity hash = SHA-256(key + installDate + source + revokedFlag + secret salt). Deleting/editing any covered value → mismatch → locked.
- Trial install date is mirrored to a signed, machine-bound breadcrumb file outside the registry; the effective date is the earliest of both, so an HKCU wipe alone cannot reset the trial.
- Activation: local, or server-backed with device binding. Server-unreachable activation is refused (no offline grant). Online keys re-validate via background heartbeat with a 7-day offline grace; revocation is enforced at next launch.

Offline key format check (validateKey)

```
key = 16 hex chars in 4 groups (dashes stripped, upper-cased)
seg0 seg1 seg2 seg3 -> take seg0, seg1, seg2
h = SHA256( seg0 + seg2 + 'KNK-SALT-2025' ).hex()
valid <=> h startswith seg1.lower()
# a self-checking key: seg1 is a checksum of seg0+seg2.
# Defeats typos, not a determined keygen (salt is in the binary).
```

License state machine

Condition at launch	Resulting state
Integrity hash mismatch / deleted	Tampered → InvalidKey (locked)
No key, within 30 days of install	Trial
No key, past 30 days	TrialExpired (key prompt or exit)
Valid local key	FullLicense (no heartbeat required)
Valid online key, last check ≤ 7 days	FullLicense (grace counting down)
Valid online key, last check > 7 days	InvalidKey until server reached
Revoked flag set (signed)	InvalidKey

Other controls

- 100% parameterised SQL (prepared statements / bound values) — no string-built queries; SQLite foreign keys enforced.
- Release-only minimal anti-debug (`IsDebuggerPresent` / `CheckRemoteDebuggerPresent`), tuned to avoid false positives on legitimate hardware/VMs.

Threat-model caveat

The key checksum salt and tamper salt ship in the binary, so the scheme deters casual tampering and keygens but is not cryptographically unbreakable. Real enforcement depends on the activation server.

9. Build, Packaging & Deployment

CMake configuration

- C++17; AUTOMOC / AUTOUIIC / AUTORCC enabled for Qt meta-object, .ui and .qrc handling.
- find_package selects Qt6 (fallback Qt5) components: Widgets, Core, Sql, PrintSupport, Network, SerialPort.
- Single executable target KeynetikPOS (WIN32 GUI subsystem); sources listed explicitly.
- Optional resources: resources/resources.qrc and a Windows .rc when present.

Deploy & package

- Post-build: windeployqt stages the exe with all Qt runtime DLLs and plugins into windeployqt_stage/.
- CPack + NSIS produces an installer (icon, Start-menu & desktop shortcuts, run-on-finish, license page); install rules ship the staged directory.
- Presets: CMakePresets (Qt) + CMakeUserPresets (Qt-Debug / Qt-Release → out/build/...).

Verified build

```
$env:PATH = 'C:\Qt\Tools\mingw1310_64\bin;' +  
            'C:\Qt\6.11.0\mingw_64\bin;' + $env:PATH  
cmake --build build\Desktop_Qt_6_11_0_MinGW_64_bit-Release --parallel 8  
# -> [100%] Built target KeynetikPOS
```

10. Configuration & Runtime Environment

Concern	Mechanism / location
Application DB	QStandardPaths::AppDataLocation + /pos_database.db (auto-created)
Business / tax / receipt config	AppSettings table (SettingsManager)
License + trial state	HKCU\Software\KeynetikSolutions\KeynetikPOS
Trial breadcrumb	Signed file under GenericConfigLocation/KeynetikSolutions
Provider credentials	QSettings (YourCompany/KeynetikPOS) Twilio + Africa's Talking
Barcode + theme prefs	QSettings (input mode, port, baud, dark mode)
Licensing endpoint	SERVER_URL constant in licensemanager.cpp (placeholder)

On first launch the database is seeded with sample categories, ~35 demo products and the default admin account. Tax defaults to disabled; currency formatting and the 16% VAT context target the Kenyan market but are configurable.

11. Engineering Notes & Technical Debt

Known trade-offs and recommended improvements, roughly in priority order:

Area	Current state	Recommendation
Testability	Singletons + logic inside MainWindow; no automated tests	Extract services behind interfaces; add a unit-test target
Money type	double / REAL with roundCents at boundaries	Store integer cents end-to-end to remove drift
MainWindow size	~1,900 LOC mixing UI and business logic	Split cart/checkout/menu controllers
Reorder levels	Hardcoded 20/100 in InventoryManager cache	Per-product columns persisted in DB
DB connection use	Some managers use the default QSqlQuery connection implicitly	Pass the connection explicitly everywhere
Parallel barcode paths	BarcodeReader (active) vs BarcodeScannerManager (legacy tables)	Consolidate on BarcodeReader
Licensing salts	Embedded in binary	Deploy the activation server; treat client as advisory

Recent hardening

The checkout double-decrement, non-atomic sale, no-op tax settings, lost payment fields, unsalted hashing, license offline-grant bypass and tamper-deletion bypass have all been resolved in v2.0.0.

12. Extension Guide

Add a database table / entity

- Add the `CREATE TABLE IF NOT EXISTS` in the owning module (`Database::createTables` or the manager's `createTableIfNotExist`).
- For new columns on existing tables, use `Database::ensureColumn(table, column, def)`.
- Expose typed DAO methods returning domain structs; keep all SQL parameterised.

Add a messaging provider

- Implement the `MessageProvider` interface (`sendMessage` + `isConfigured`).
- Register it with `ScheduleManager::registerProvider()`; surface credentials in `SettingsDialog` and persist via `QSettings`.

Add a permission / role rule

- Extend the `Permission` enum in `user.h` and describe it in `RoleManager`.
- Update `RoleManager::hasPermission` allow-lists; gate UI with `checkPermission()` and re-check inside the manager method.

Add a report or analytics widget

- Add aggregation in `AnalyticsManager/Database`; build a `QWidget` chart in `enhancedanalytics` and register it in `EnhancedAnalyticsDialog`, or add a report type to `ReportsDialog` with CSV/PDF export.

Testing & QA recommendations

- High-value regression cases: checkout atomicity (force a mid-loop failure → expect rollback, intact cart, unchanged stock); oversell guard under low stock; tax inclusive vs exclusive vs disabled; refund double-submit; legacy-hash login upgrade; trial reset after registry wipe (breadcrumb should hold).
- To make these unit-testable, inject the DB connection / managers instead of using the singletons, and run against an in-memory SQLite database (`':memory:'`).
- Add a `QtTest` target in `CMake` (`enable_testing` + `add_test`) for CI.

Glossary

Term	Meaning
DAO	Data-access object — typed methods wrapping SQL
RBAC	Role-based access control (Admin/Manager/Cashier/Viewer)
PBKDF2	Password-Based Key Derivation Function 2 (salted, iterated hash)
Z-Report	End-of-day/shift sales & cash reconciliation report
X-Report	Mid-shift read-only sales snapshot (no reset)
Keyboard wedge	USB scanner that emulates a keyboard typing the barcode
Breadcrumb	Signed out-of-registry file anchoring the trial install date
windeployqt	Qt tool that bundles runtime DLLs/plugins next to the exe

KeynetikPOS v2.0.0 — Engineering Documentation. Generated from the application source; reflects the architecture at the time of writing.